In [1]:

```
# Import required libraries

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sb
```

In [2]:

```
# Import the Placement dataset

dataset = pd.read_csv('Placement.csv')

# Display the dataset
dataset
```

Out[2]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8
...	...	...	...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60	Comm&Mgmt	No	91.0
211	212	M	58.00	Others	60.00	Others	Science	72.00	Sci&Tech	No	74.0
212	213	M	67.00	Others	67.00	Others	Commerce	73.00	Comm&Mgmt	Yes	59.0
213	214	F	74.00	Others	66.00	Others	Commerce	58.00	Comm&Mgmt	No	70.0
214	215	M	62.00	Central	58.00	Others	Science	53.00	Comm&Mgmt	No	89.0

215 rows × 15 columns

In [3]:

```
# Number of rows and columns

print("The Number of Rows:", dataset.shape[0])
print("The Number of Columns:", dataset.shape[1])
```

```
The Number of Rows: 215
The Number of Columns: 15
```

In [4]:

```
dataset.shape
```

Out[4]:

```
(215, 15)
```

In [5]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Display first 5 rows

dataset.head()
```

Out[5]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p
0	1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	No	55.0
1	2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Yes	86.5
2	3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	No	75.0
3	4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	No	66.0
4	5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	No	96.8

In [6]:

```
# Display last 5 rows

dataset.tail()
```

Out[6]:

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	etest_p
210	211	M	80.6	Others	82.0	Others	Commerce	77.6	Comm&Mgmt	No	91.0
211	212	M	58.0	Others	60.0	Others	Science	72.0	Sci&Tech	No	74.0
212	213	M	67.0	Others	67.0	Others	Commerce	73.0	Comm&Mgmt	Yes	59.0
213	214	F	74.0	Others	66.0	Others	Commerce	58.0	Comm&Mgmt	No	70.0
214	215	M	62.0	Central	58.0	Others	Science	53.0	Comm&Mgmt	No	89.0

In [7]:

```
# Check column names

dataset.columns
```

Out[7]:

```
Index(['sl_no', 'gender', 'ssc_p', 'ssc_b', 'hsc_p', 'hsc_b', 'hsc_s',
       'degree_p', 'degree_t', 'workex', 'etest_p', 'specialisation', 'mba_p',
       'status', 'salary'],
      dtype='str')
```

In [8]:

```
# Check data types

dataset.dtypes
```

Out[8]:

```
sl_no             int64
gender              str
ssc_p           float64
ssc_b               str
hsc_p           float64
hsc_b               str
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
hsc_s                str
degree_p           float64
degree_t             str
workex               str
etest_p            float64
specialisation       str
mba_p              float64
status               str
salary             float64
dtype: object
```

In [9]:

```
# Display basic information about the dataset

dataset.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 215 entries, 0 to 214
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   sl_no           215 non-null    int64
 1   gender          215 non-null    str
 2   ssc_p           215 non-null    float64
 3   ssc_b           215 non-null    str
 4   hsc_p           215 non-null    float64
 5   hsc_b           215 non-null    str
 6   hsc_s           215 non-null    str
 7   degree_p        215 non-null    float64
 8   degree_t        215 non-null    str
 9   workex          215 non-null    str
 10  etest_p         215 non-null    float64
 11  specialisation  215 non-null    str
 12  mba_p           215 non-null    float64
 13  status          215 non-null    str
 14  salary          148 non-null    float64
dtypes: float64(6), int64(1), str(8)
memory usage: 35.0 KB
```

In [10]:

```
# Check missing values in each column

dataset.isnull().sum()
```

Out[10]:

```
sl_no             0
gender            0
ssc_p             0
ssc_b             0
hsc_p             0
hsc_b             0
hsc_s             0
degree_p          0
degree_t          0
workex            0
etest_p           0
specialisation    0
mba_p             0
status            0
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
salary            67
dtype: int64
```

In [11]:

```
# Statistical description of numerical columns

dataset.describe()
```

Out[11]:

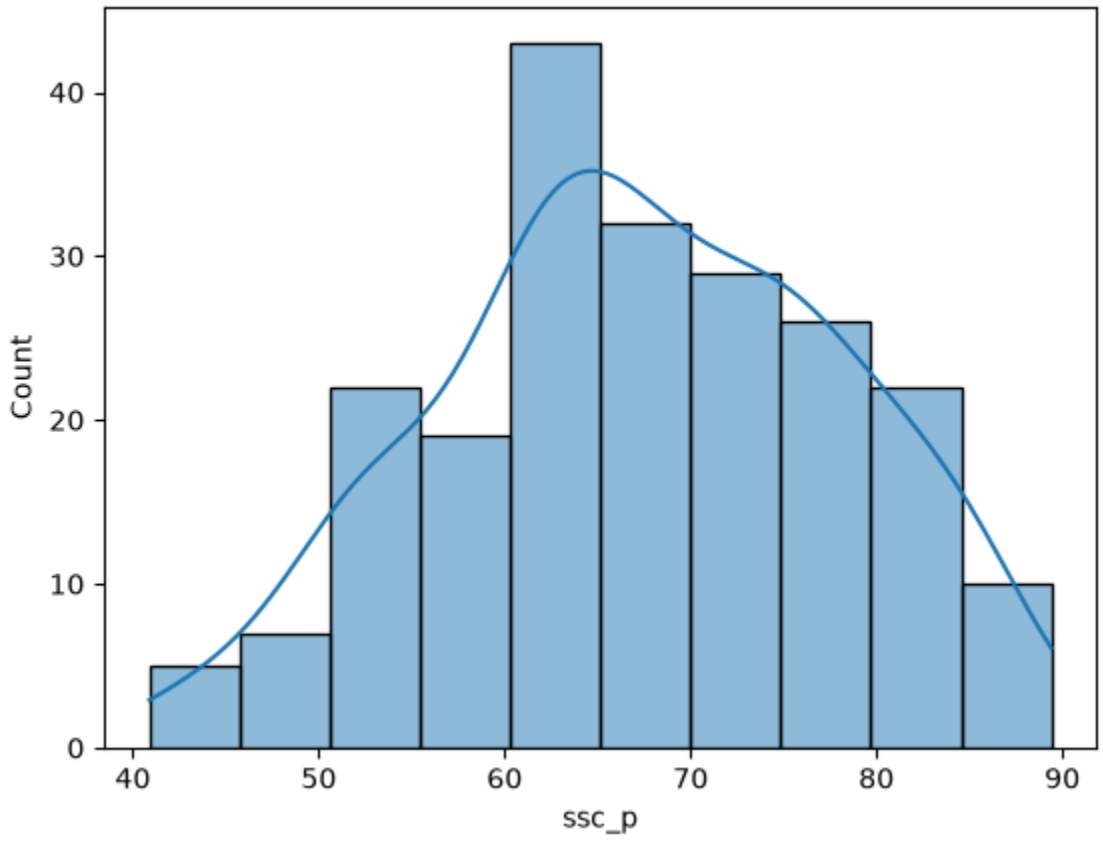
	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
count	215.000000	215.000000	215.000000	215.000000	215.000000	215.000000	148.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558	62.278186	288655.405405
std	62.209324	10.827205	10.897509	7.358743	13.275956	5.833385	93457.452420
min	1.000000	40.890000	37.000000	50.000000	50.000000	51.210000	200000.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000	57.945000	240000.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000	62.000000	265000.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000	66.255000	300000.000000
max	215.000000	89.400000	97.700000	91.000000	98.000000	77.890000	940000.000000

In [12]:

```
# Univariate Analysis
# Distribution of SSC percentage

sb.histplot(dataset['ssc_p'], kde=True)

plt.show()
```



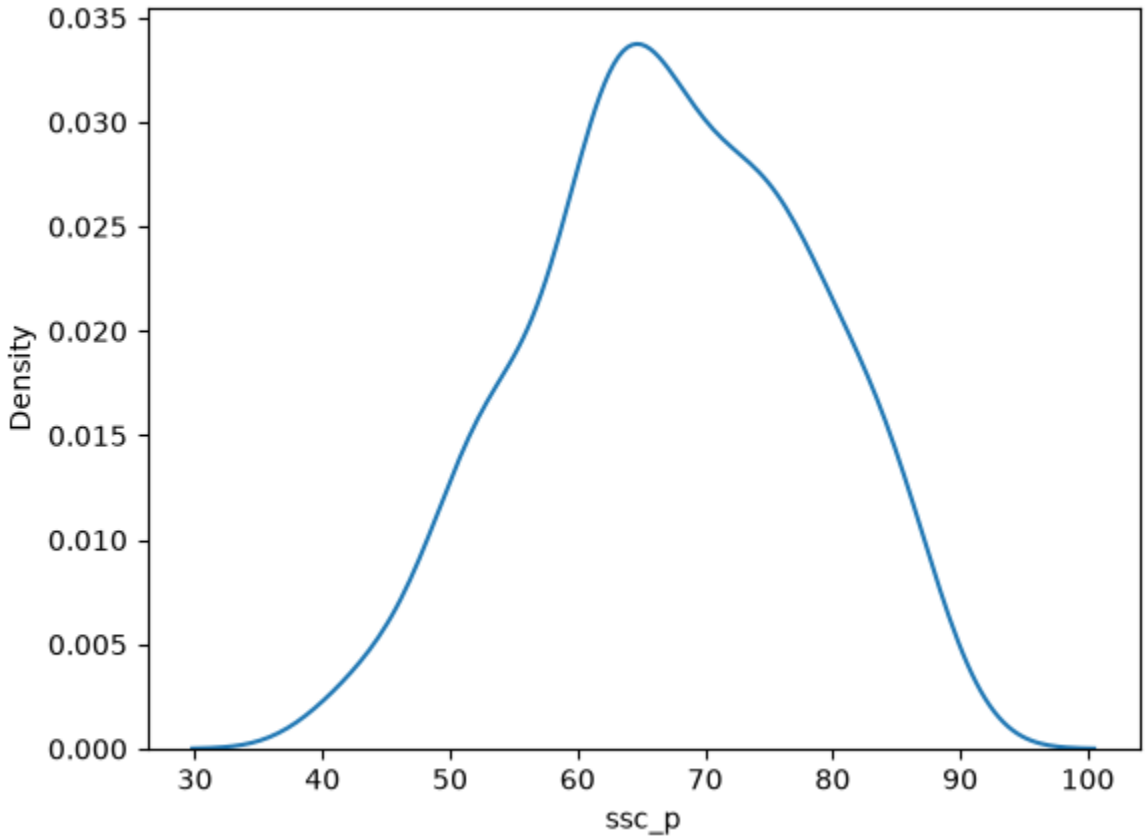
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [13]:

```
# KDE plot for SSC percentage

sb.kdeplot(data=dataset, x='ssc_p')

plt.show()
```



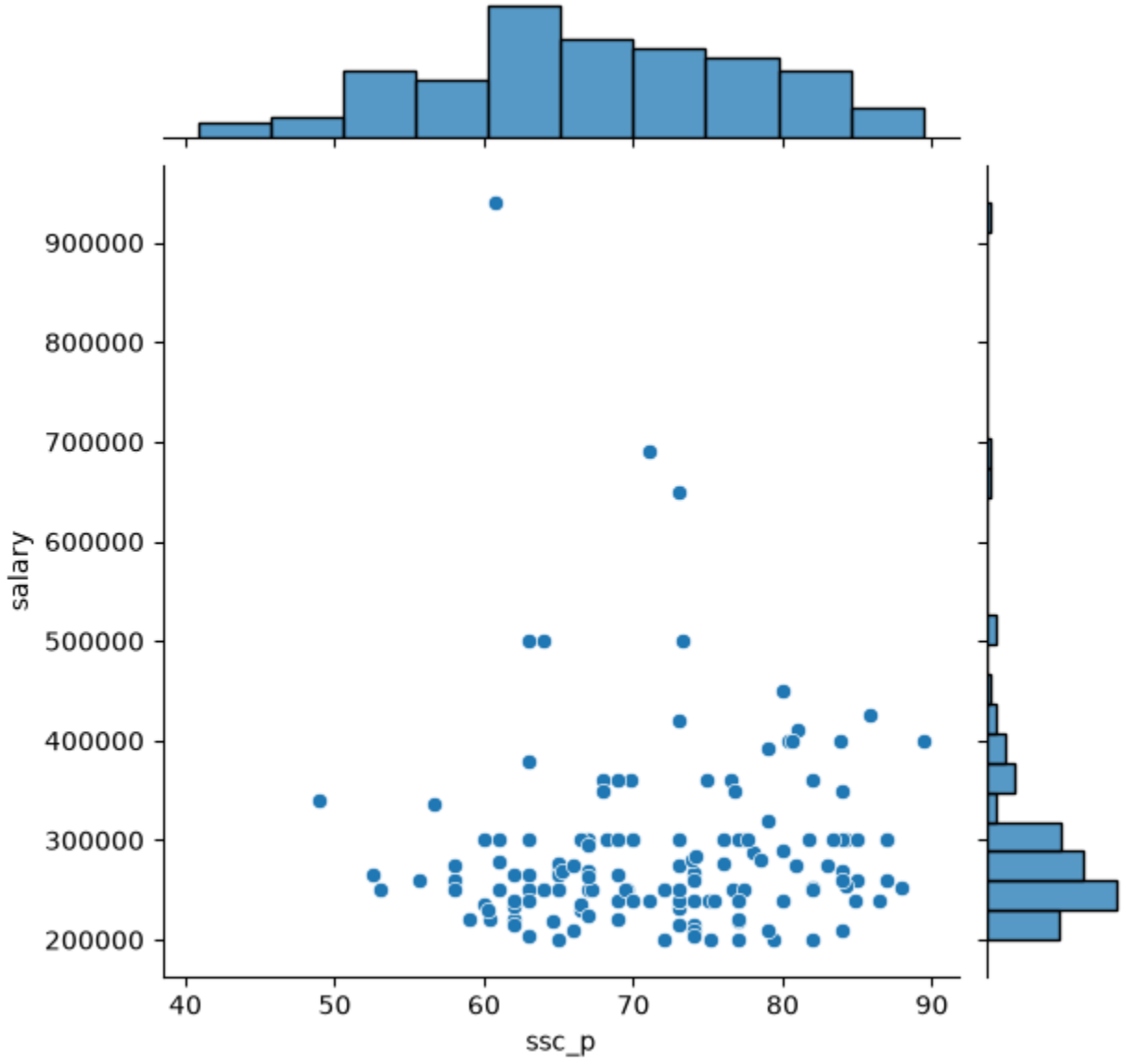
In [14]:

```
sb.jointplot(x='ssc_p', y='salary', data=dataset)
```

Out[14]:

```
<seaborn.axisgrid.JointGrid at 0x2a52e9b2e40>
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



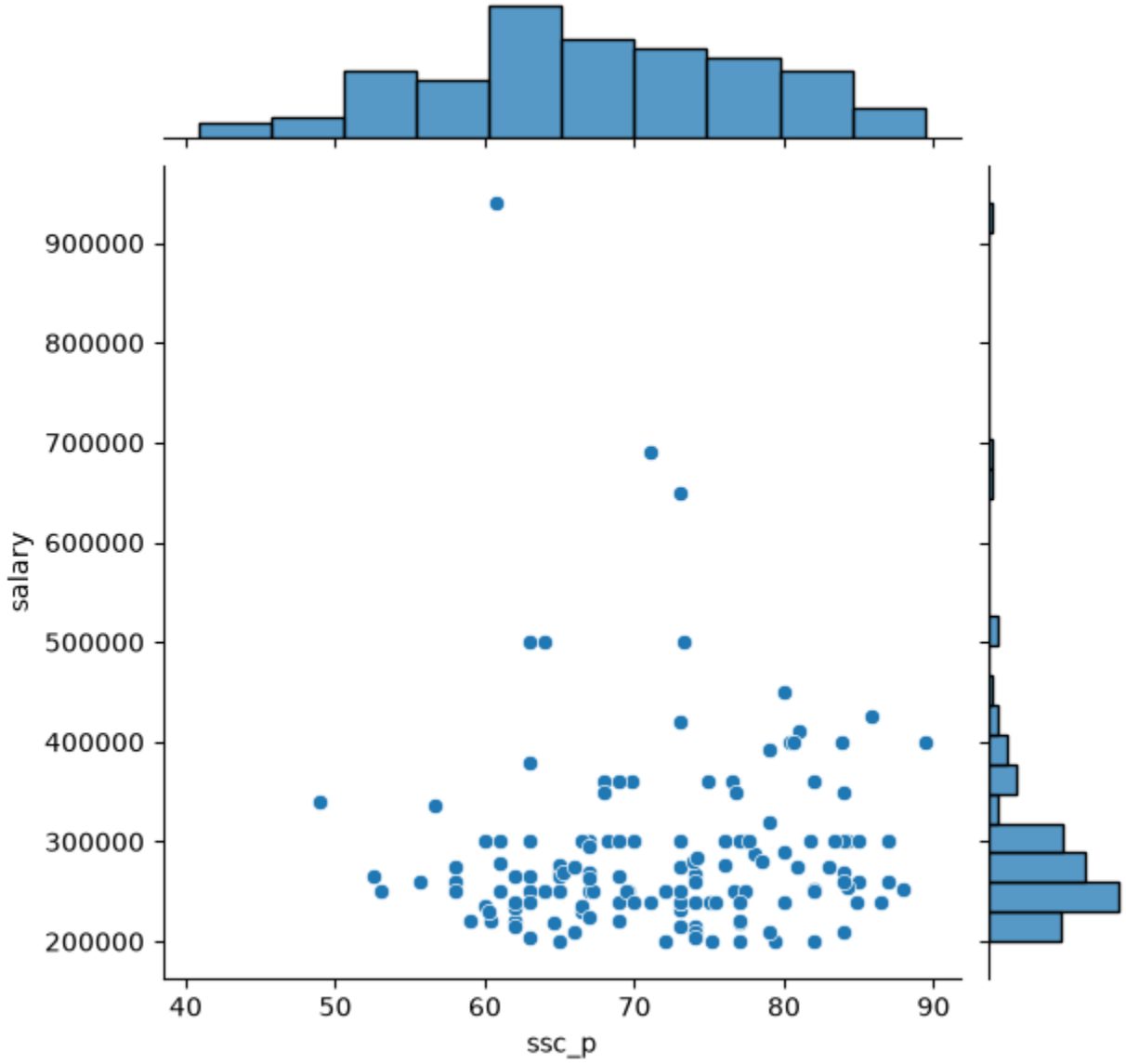
In [15]:

```
# Bivariate Analysis
# Relationship between SSC percentage and Salary

sb.jointplot(
    x='ssc_p',
    y='salary',
    data=dataset
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



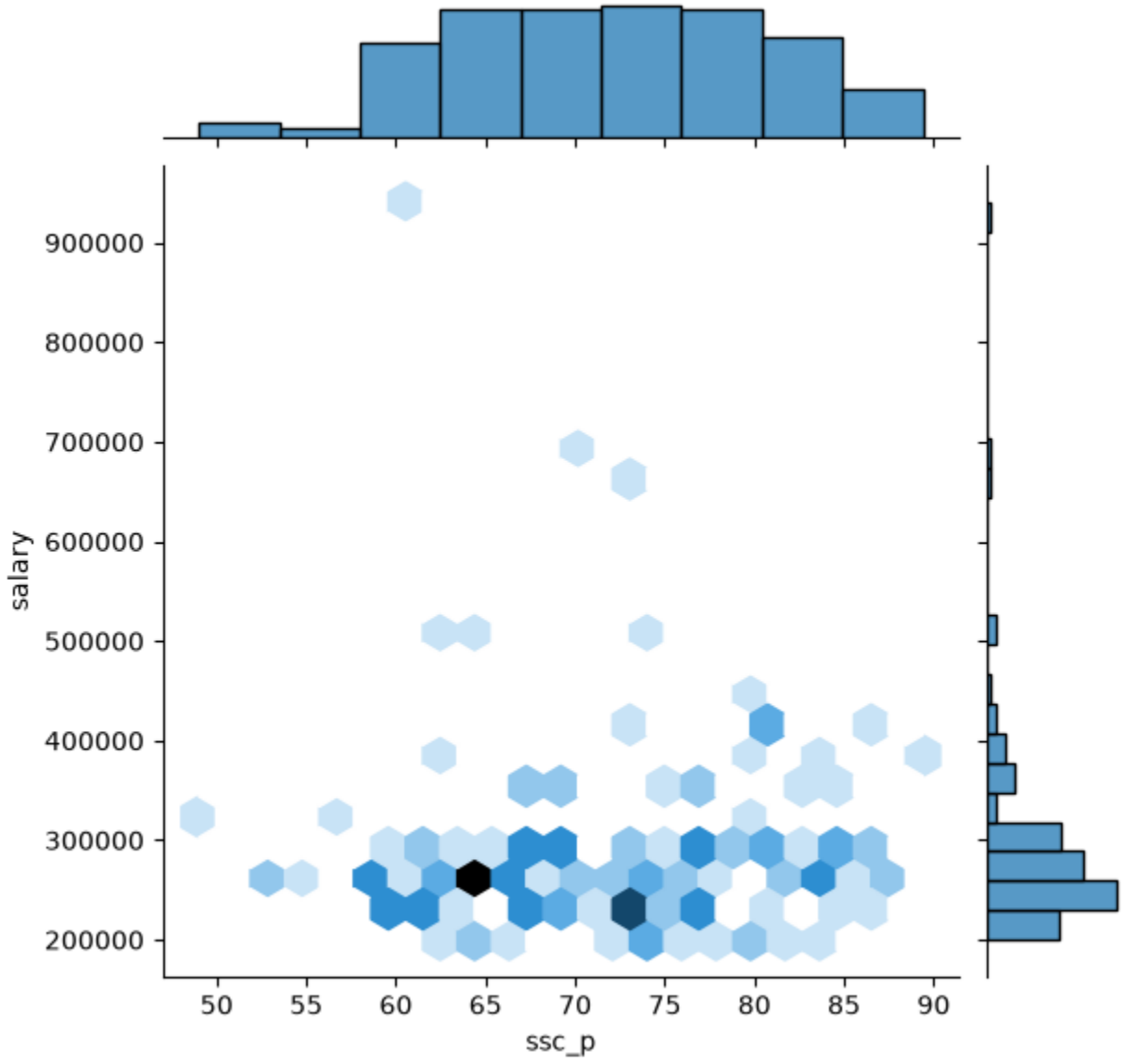
In [16]:

```
# Joint plot using hexbin

sb.jointplot(
    x='ssc_p',
    y='salary',
    data=dataset,
    kind='hex'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



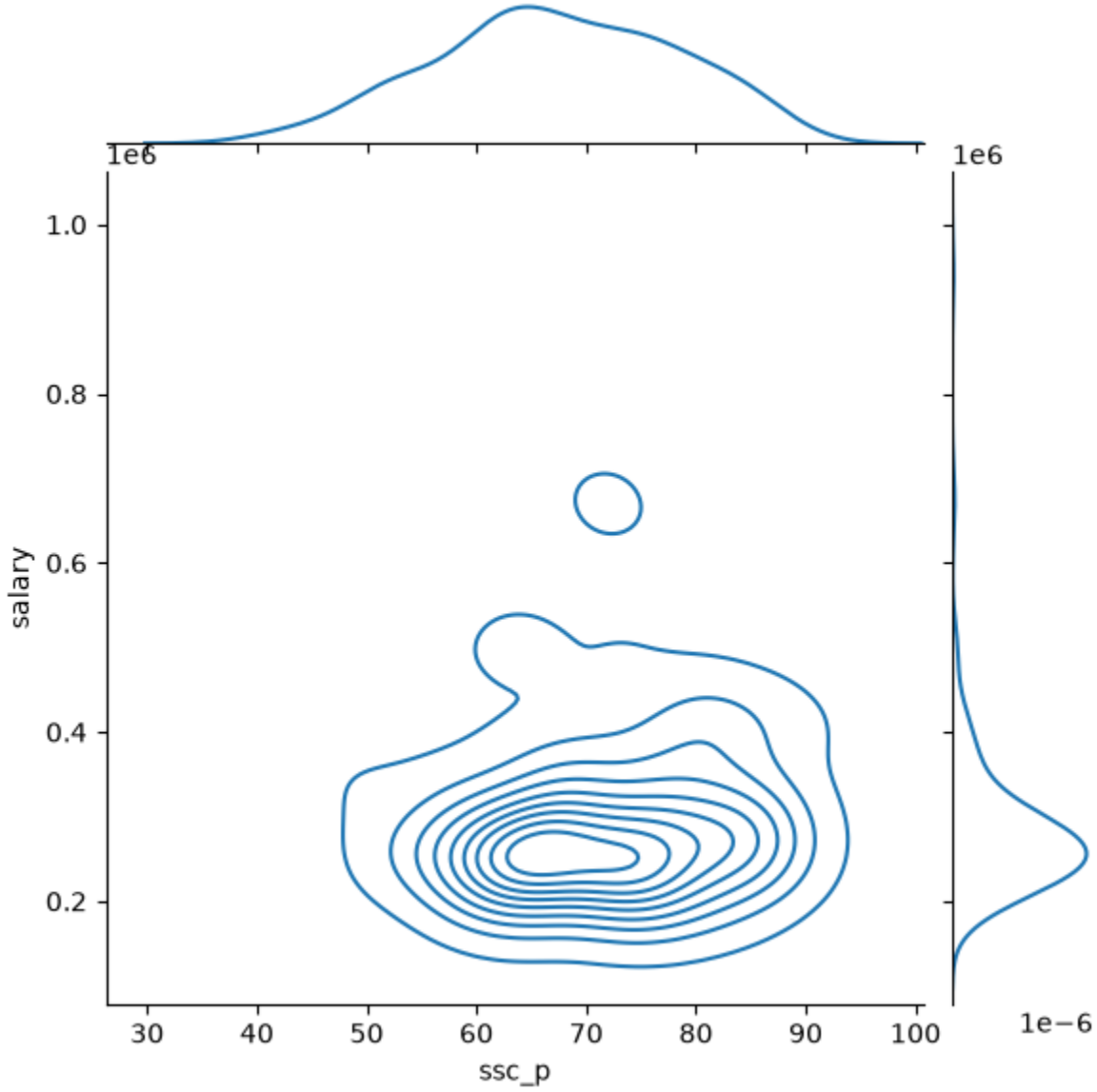
In [17]:

```
# Joint plot using KDE

sb.jointplot(
    x='ssc_p',
    y='salary',
    data=dataset,
    kind='kde'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



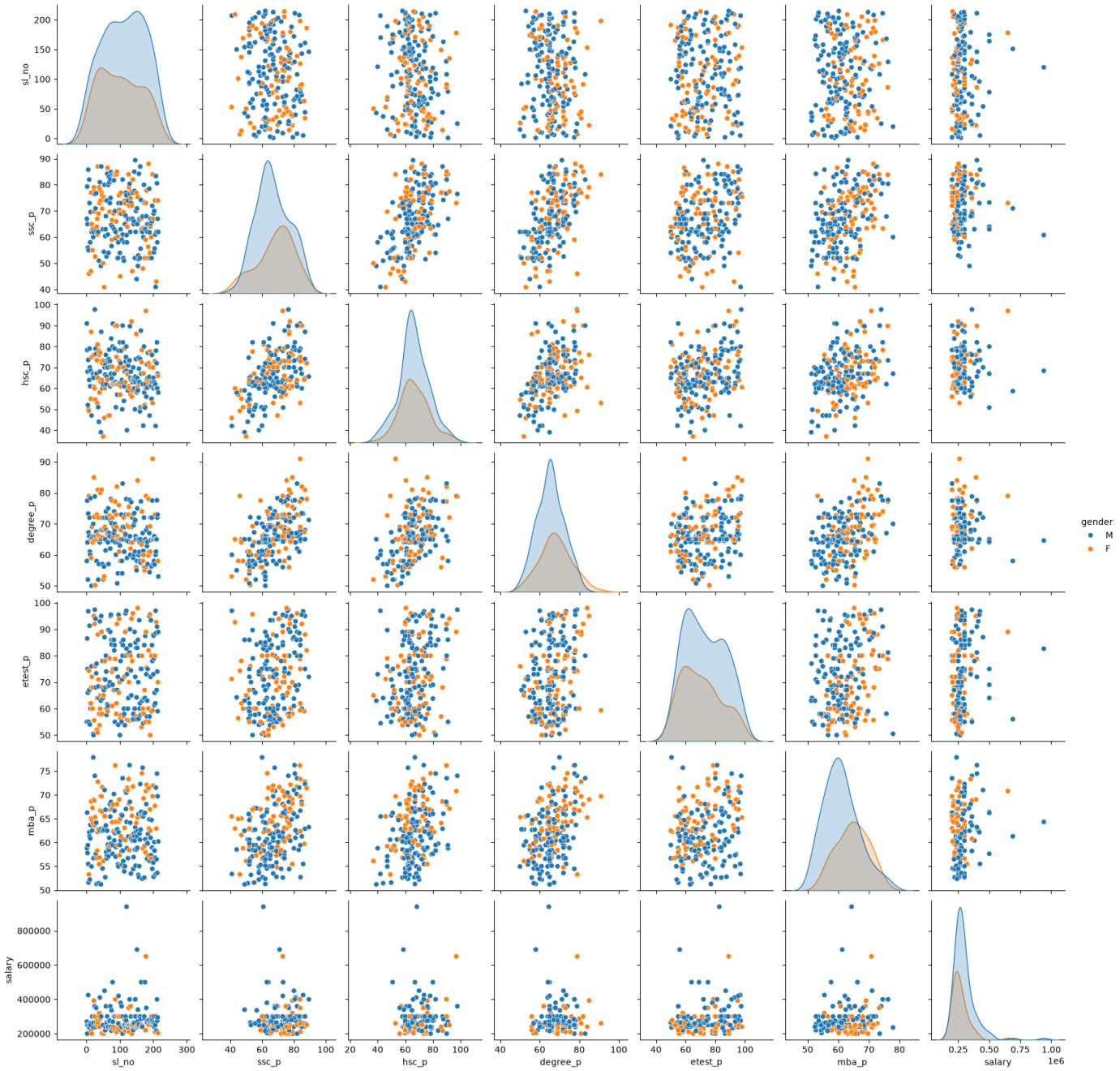
In [18]:

```
# Pair Plot

sb.pairplot(
    dataset,
    hue='gender',
    diag_kind='kde',
    kind='scatter'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



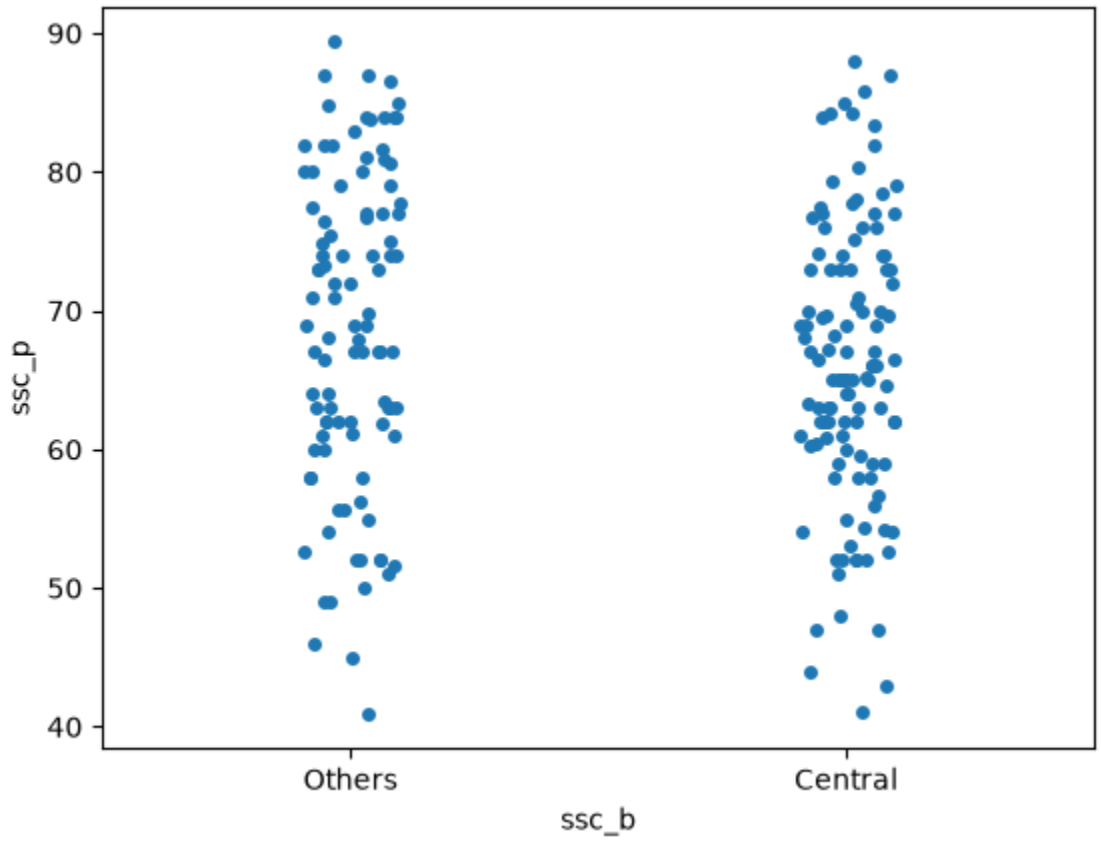
In [19]:

```
# Strip Plot

sb.stripplot(
    x='ssc_b',
    y='ssc_p',
    data=dataset
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



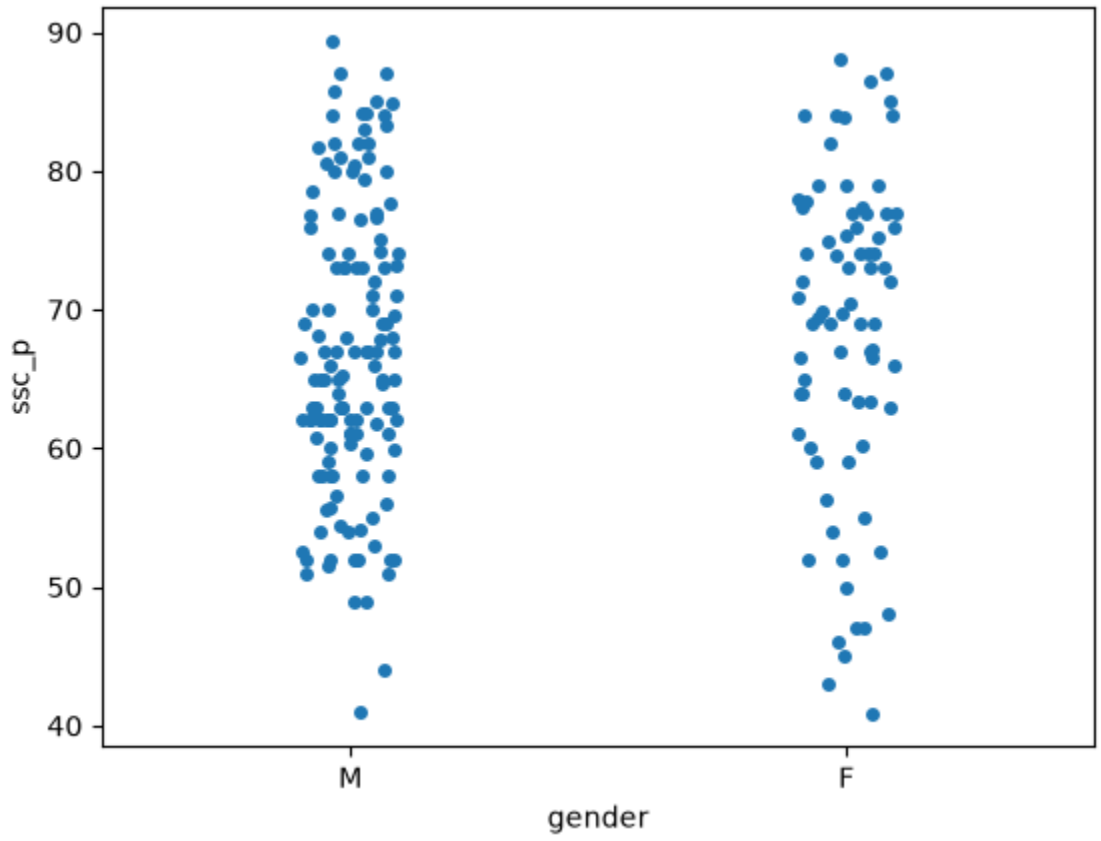
In [20]:

```
# Strip plot based on gender

sb.stripplot(
    x='gender',
    y='ssc_p',
    data=dataset,
    jitter=True
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



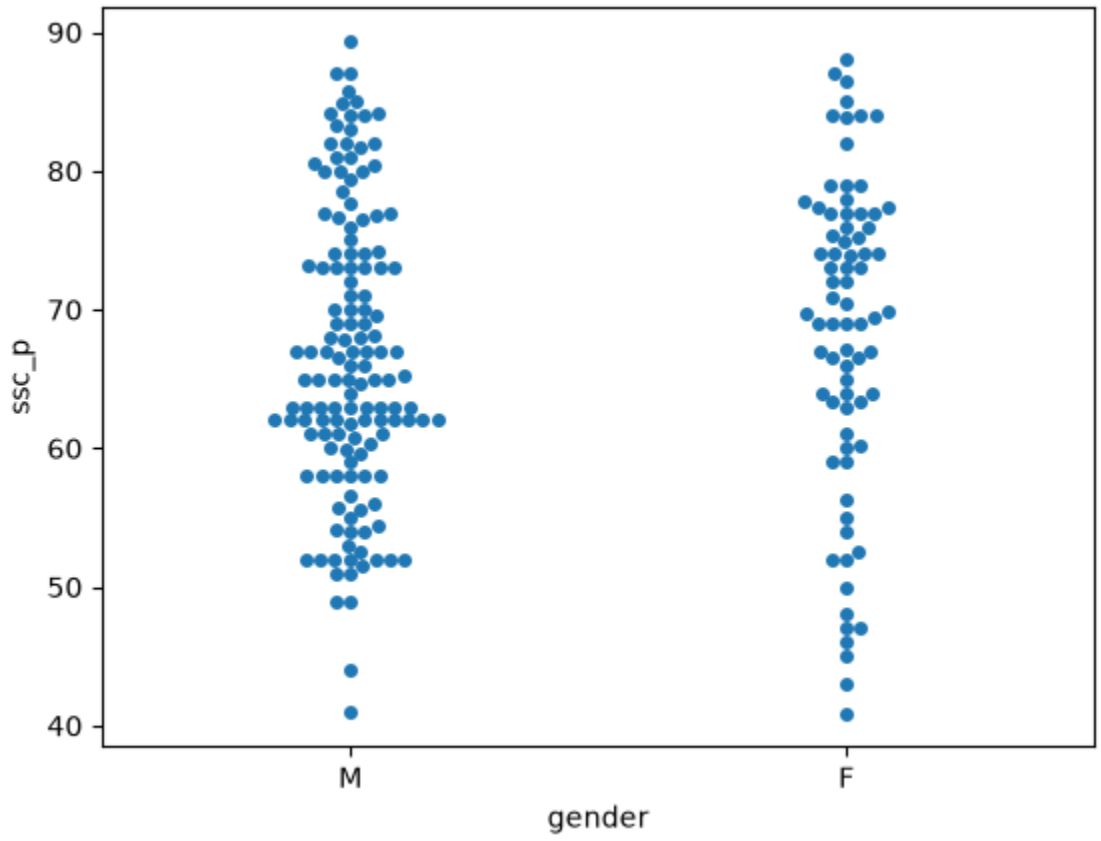
In [21]:

```
# Swarm Plot

sb.swarmplot(
    x='gender',
    y='ssc_p',
    data=dataset
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



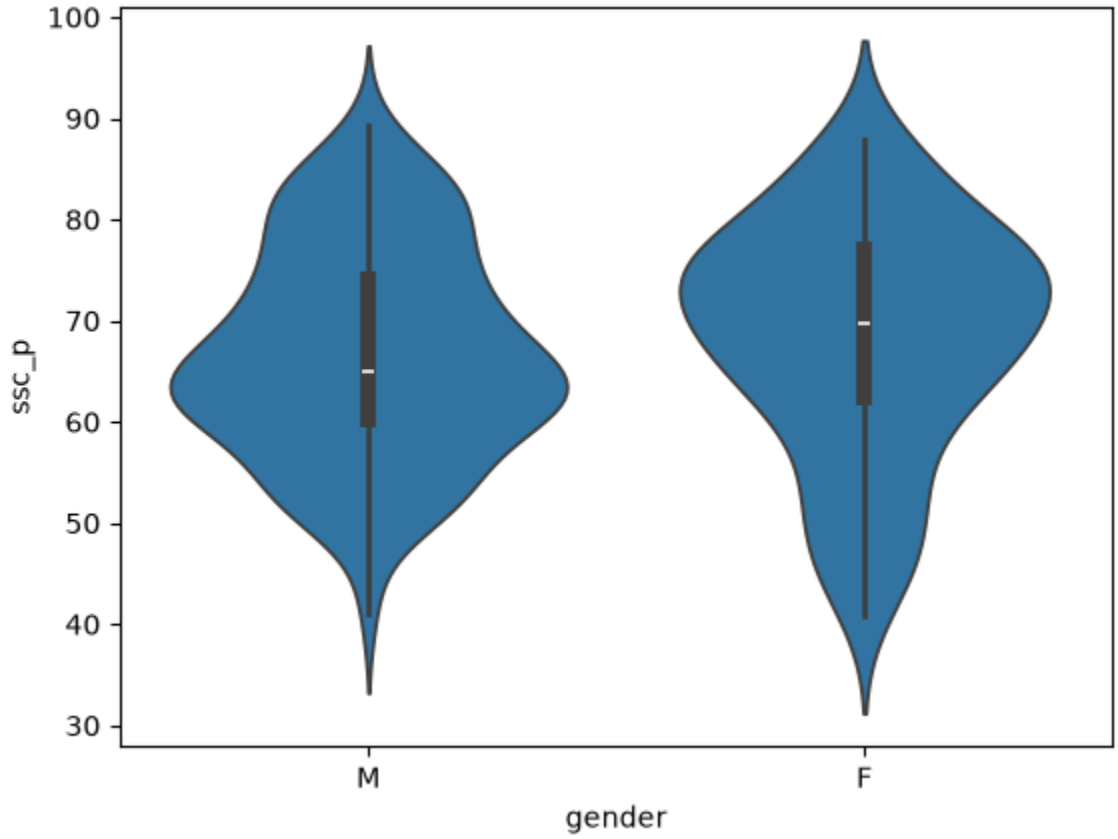
In [22]:

```
# Violin Plot

sb.violinplot(
    x='gender',
    y='ssc_p',
    data=dataset
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



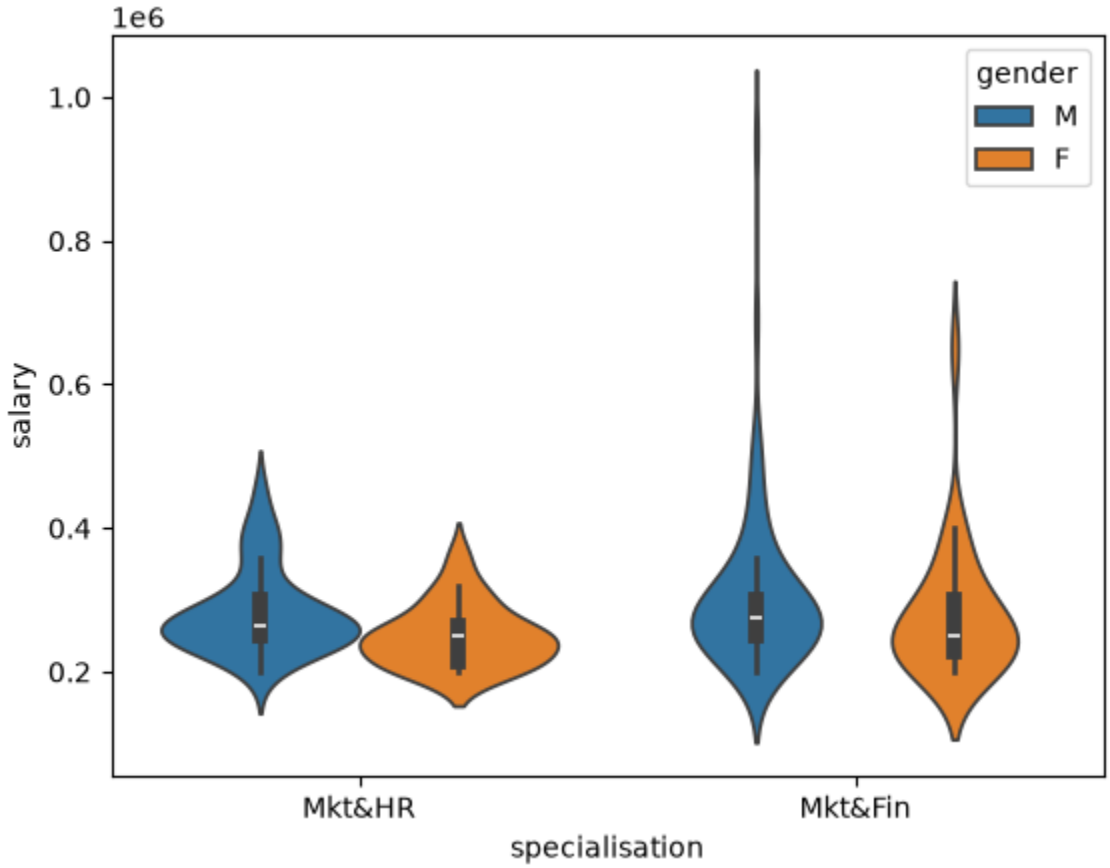
In [23]:

```
# Violin plot of salary based on specialisation and gender

sb.violinplot(
    x='specialisation',
    y='salary',
    data=dataset,
    hue='gender'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



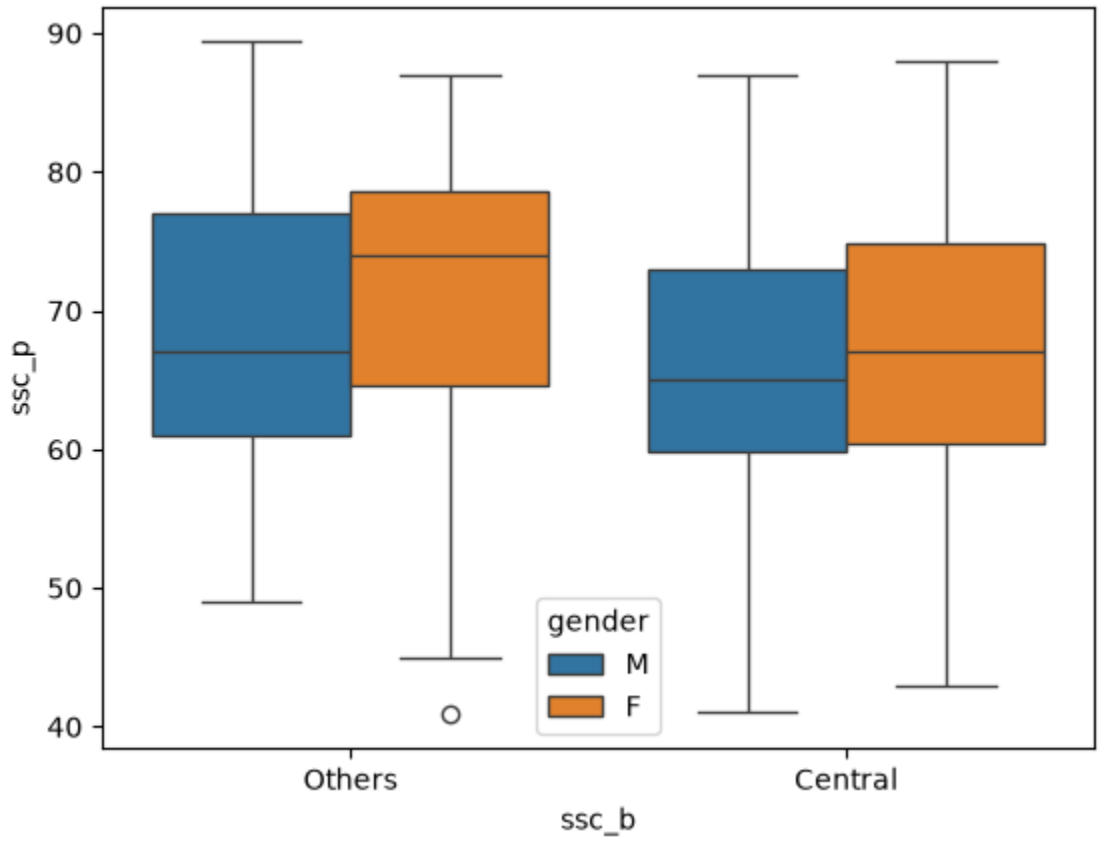
In [24]:

```
# Box Plot

sb.boxplot(
    x='ssc_b',
    y='ssc_p',
    data=dataset,
    hue='gender'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



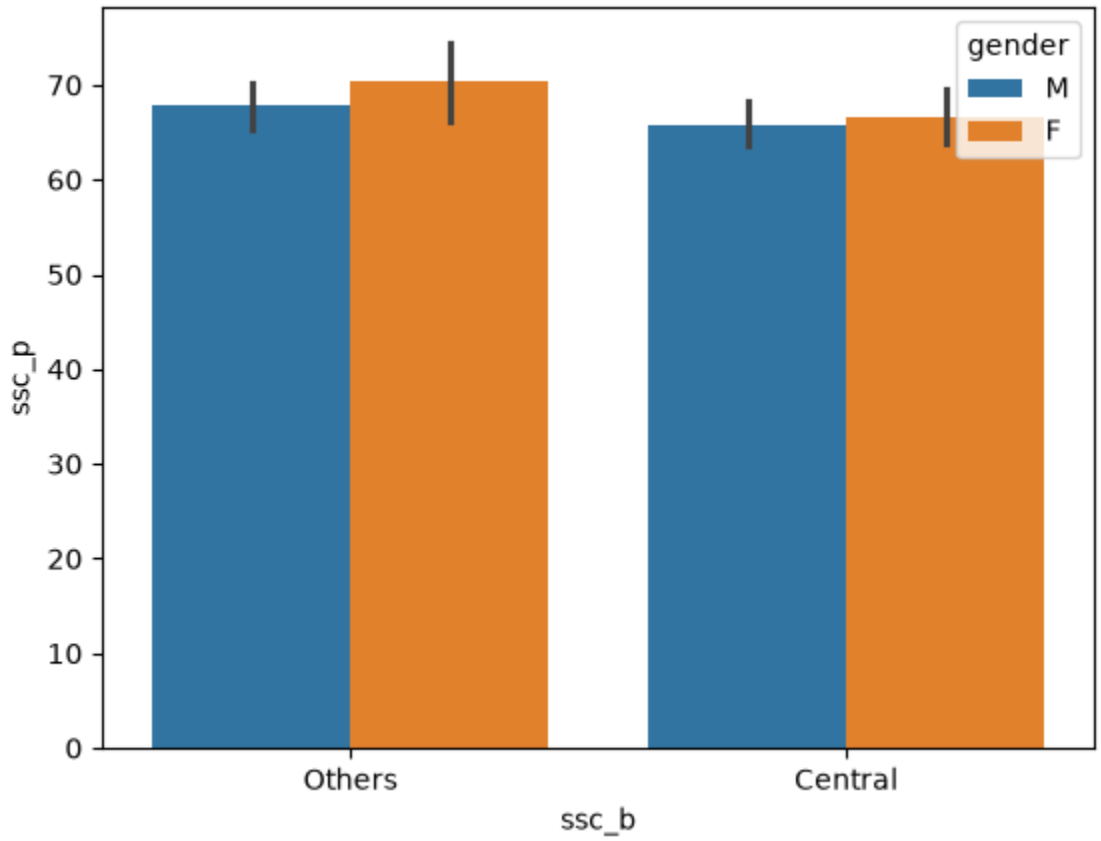
In [25]:

```
# Bar Plot

sb.barplot(
    x='ssc_b',
    y='ssc_p',
    data=dataset,
    hue='gender'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



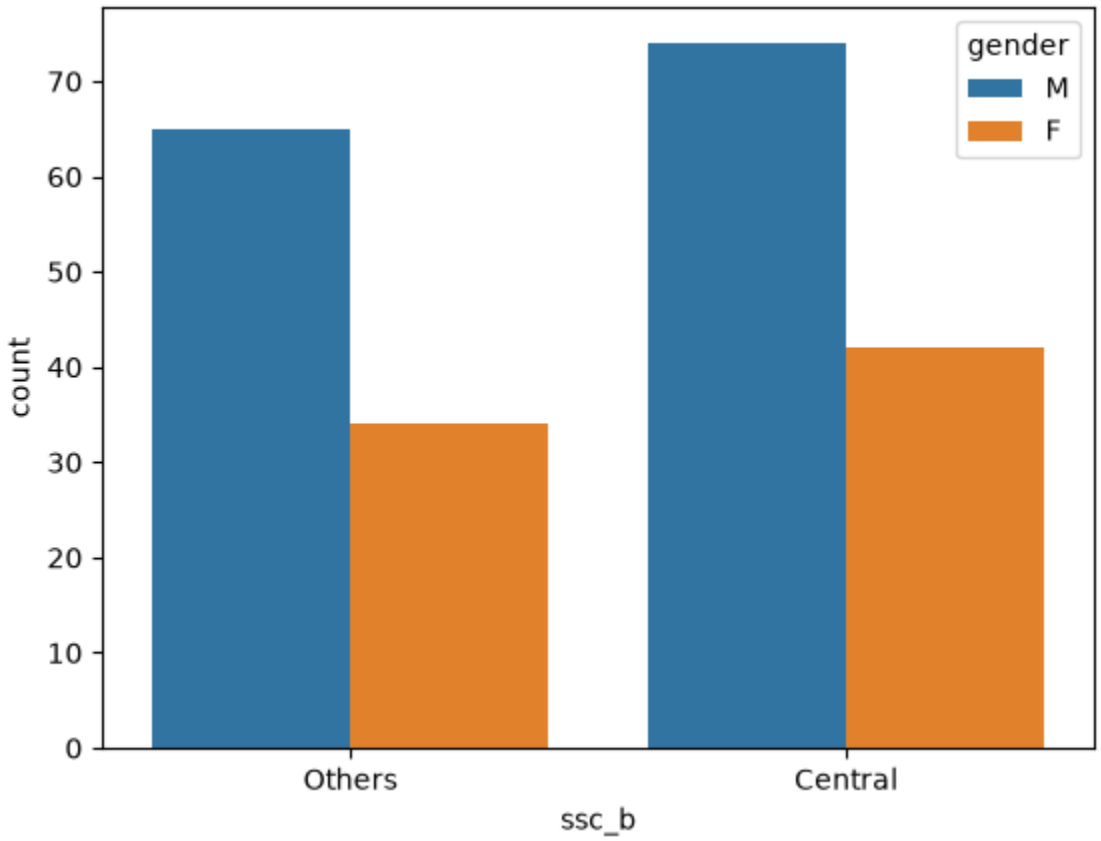
In [26]:

```
# Count Plot

sb.countplot(
    x='ssc_b',
    data=dataset,
    hue='gender'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



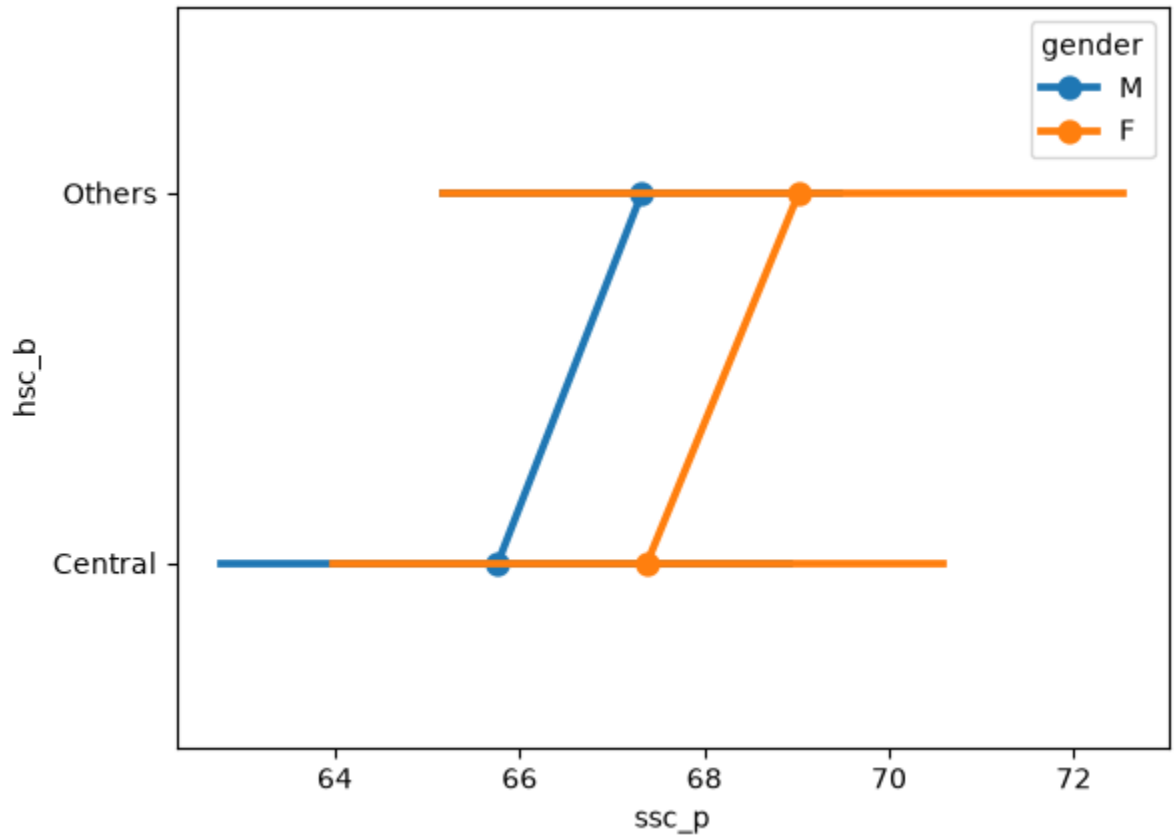
In [27]:

```
# Point Plot

sb.pointplot(
    x='ssc_p',
    y='hsc_b',
    data=dataset,
    hue='gender'
)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

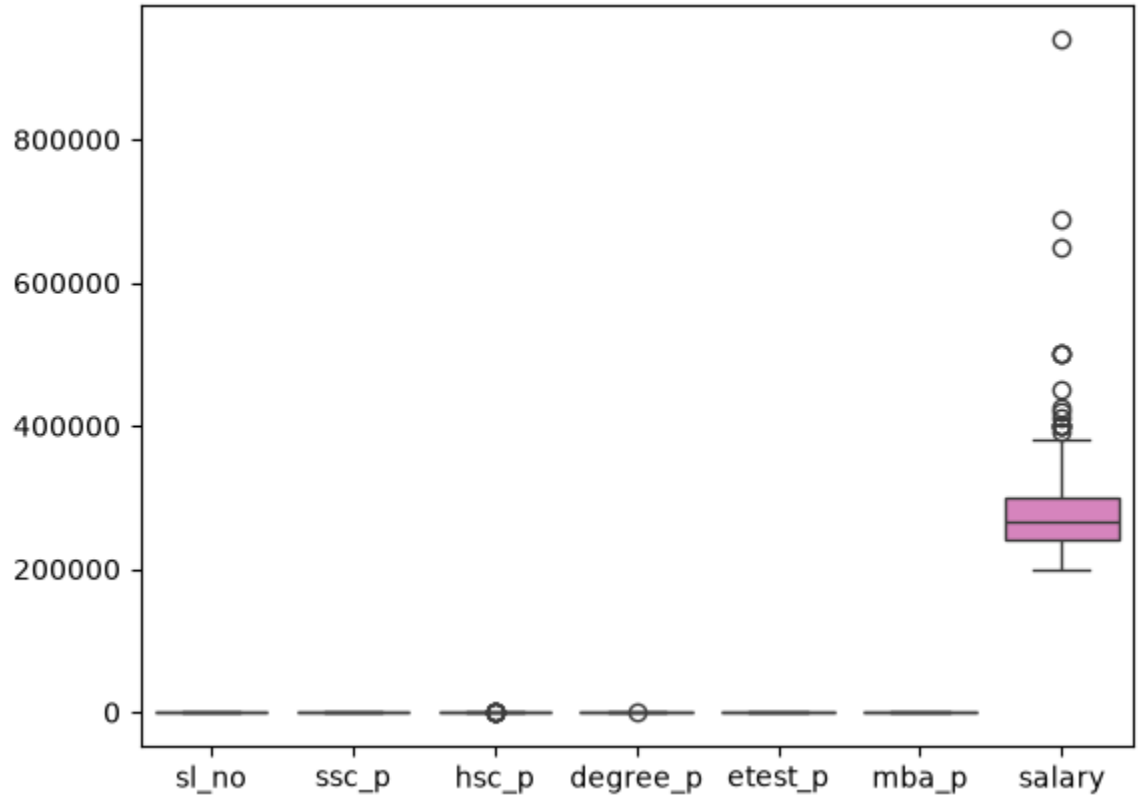


In [28]:

```
# Overall Box Plot

sb.boxplot(data=dataset)

plt.show()
```



In [29]:

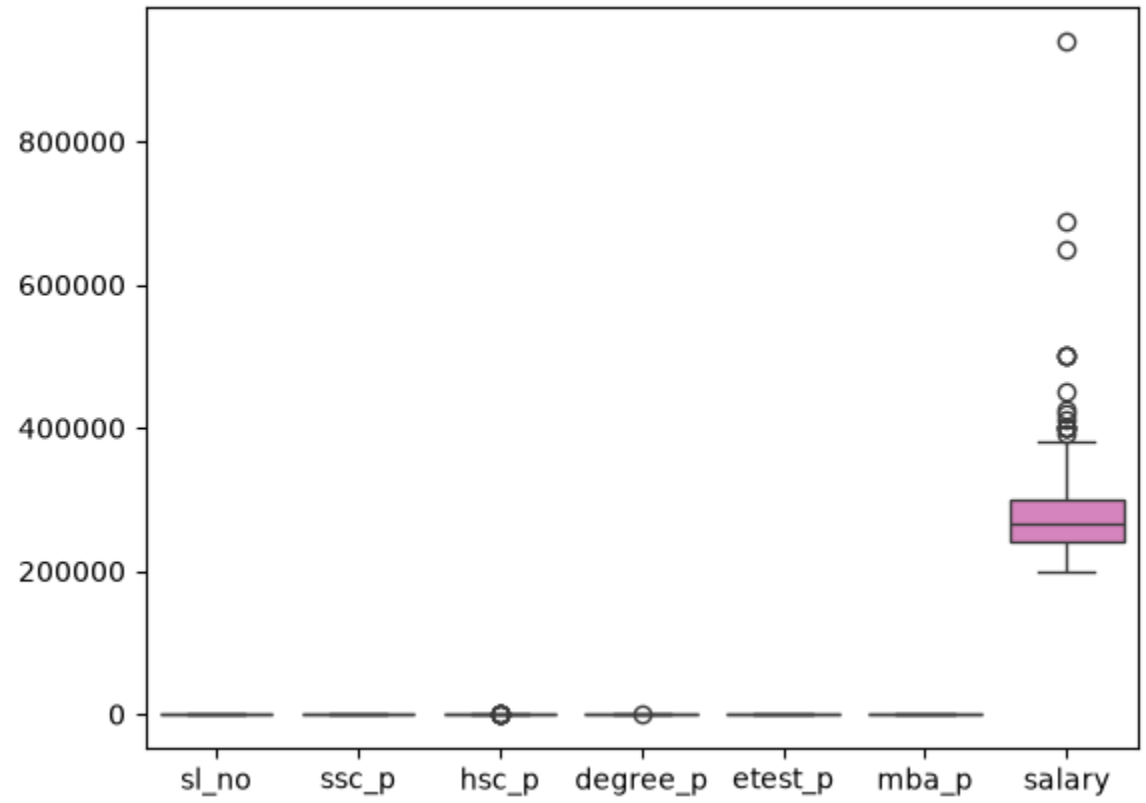
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Box plot for numerical columns only

numeric_data = dataset.select_dtypes(include=np.number)

sb.boxplot(data=numeric_data)

plt.show()
```



In [34]:

```
# Create sample data

data = {
    "Product": ["Laptop", "Mobile", "Tablet", "Headphone", "Watch"],
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [500, 700, 450, 800, 600],
    "Profit": [100, 150, 80, 200, 120],
    "Marketing": [50, 70, 40, 90, 60]
}
```

In [35]:

```
# Create DataFrame

dataset = pd.DataFrame(data)

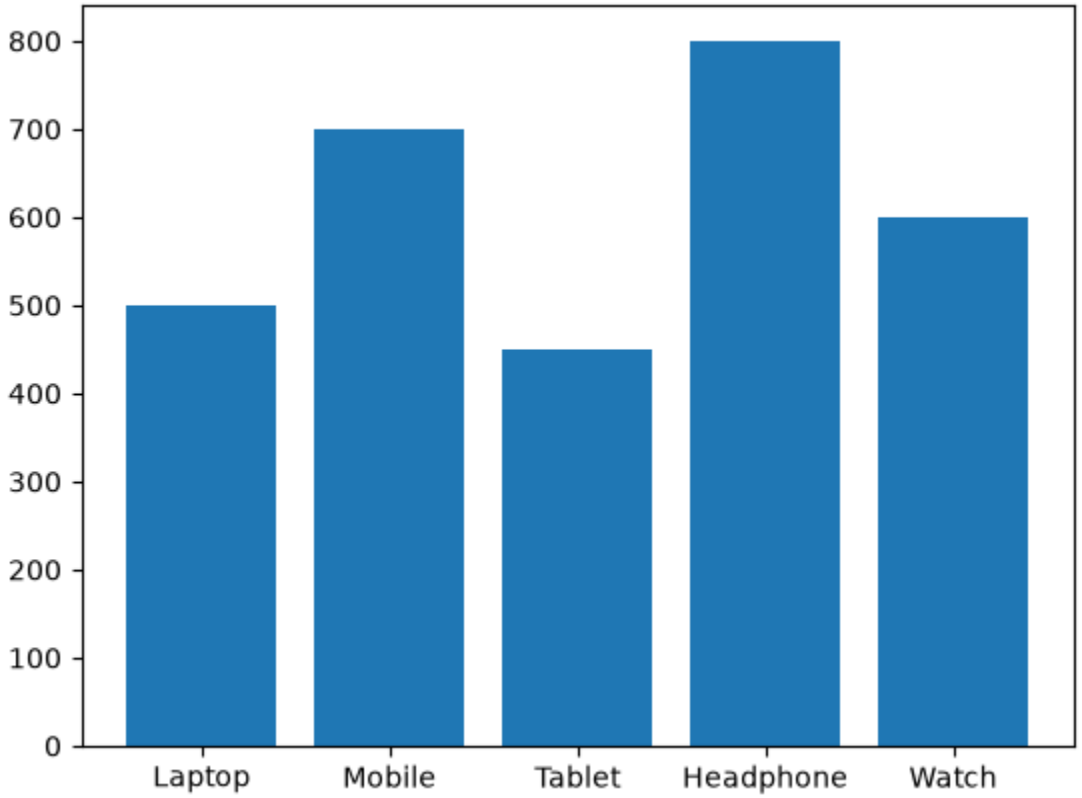
# Display the dataset
print(dataset)
```

```
     Product Month  Sales  Profit  Marketing
0     Laptop   Jan    500     100         50
1     Mobile   Feb    700     150         70
2     Tablet   Mar    450      80         40
3  Headphone   Apr    800     200         90
4      Watch   May    600     120         60
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [36]:

```
plt.bar(dataset["Product"], dataset["Sales"])
plt.show()
```



In [37]:

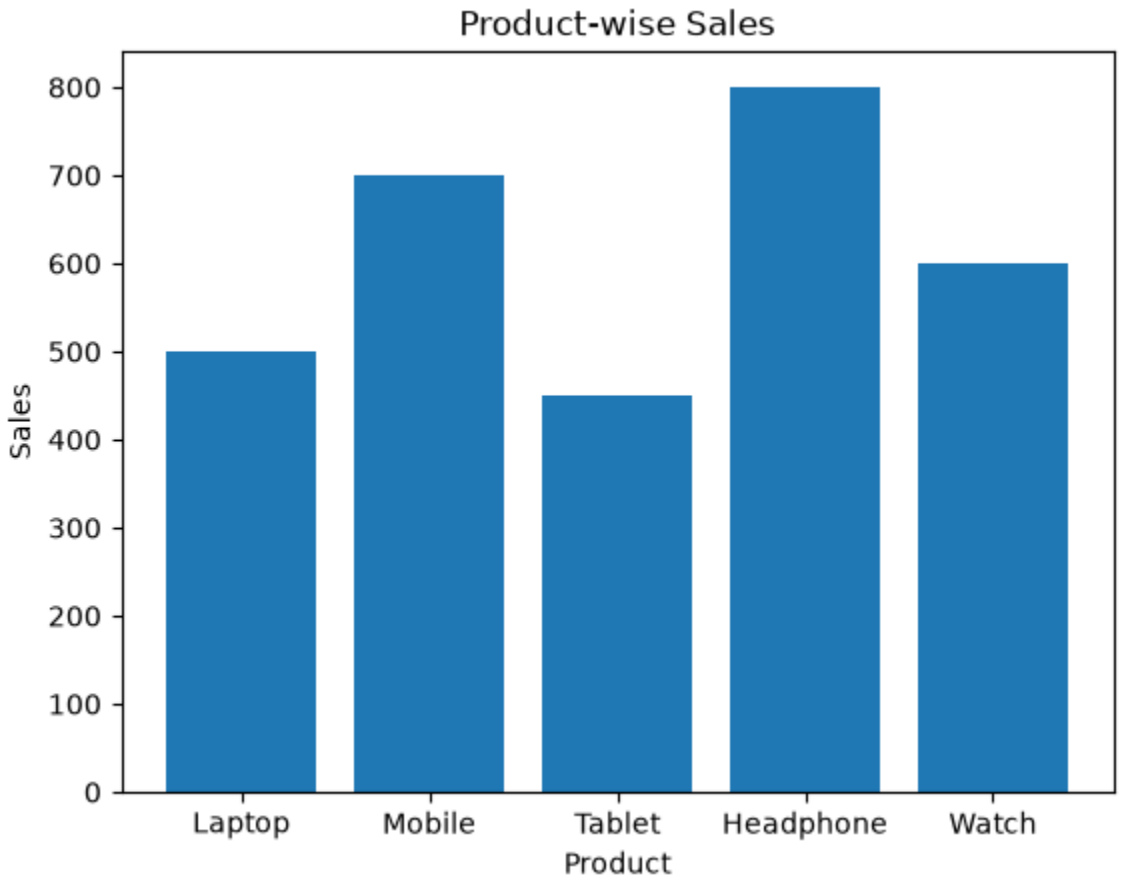
```
# Bar Chart
# Shows sales for each product

plt.bar(dataset["Product"], dataset["Sales"])

plt.xlabel("Product")
plt.ylabel("Sales")
plt.title("Product-wise Sales")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [38]:

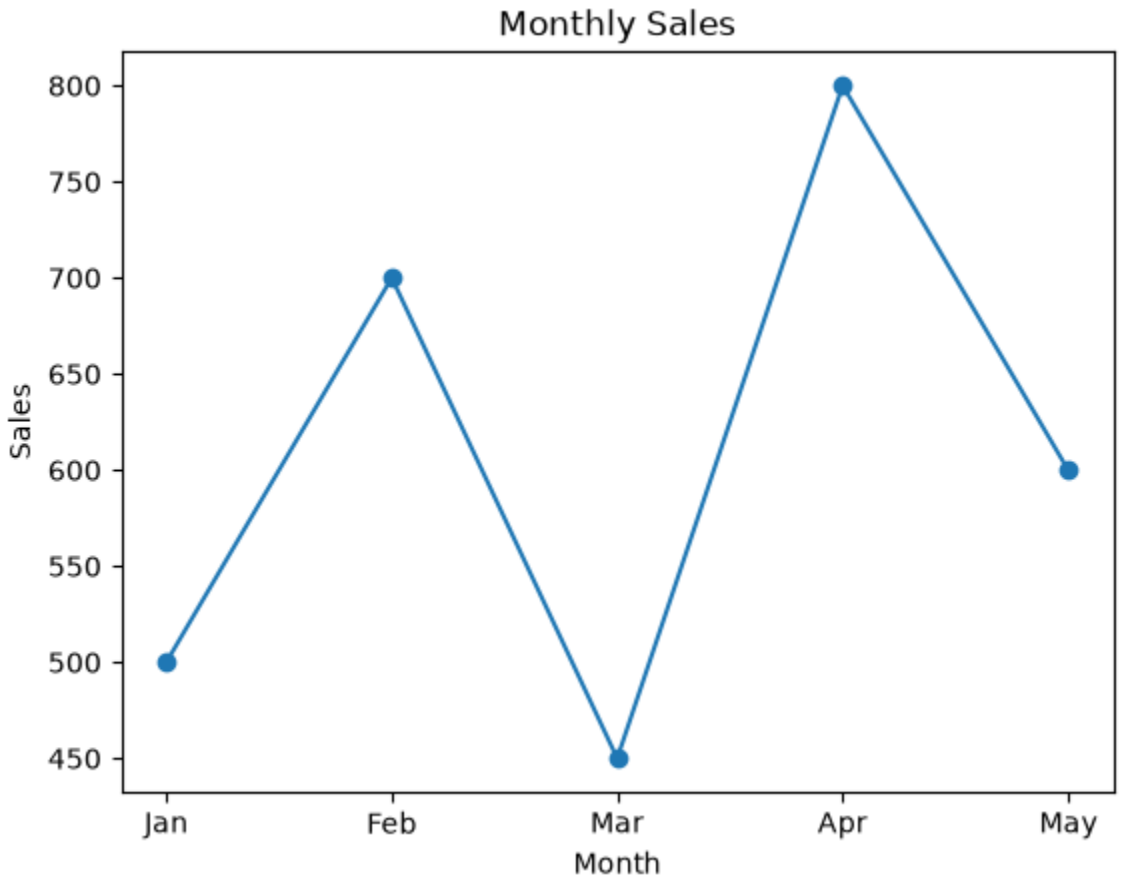
```
# Line Chart
# Shows sales trend across months

plt.plot(
    dataset["Month"],
    dataset["Sales"],
    marker='o'
)

plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [39]:

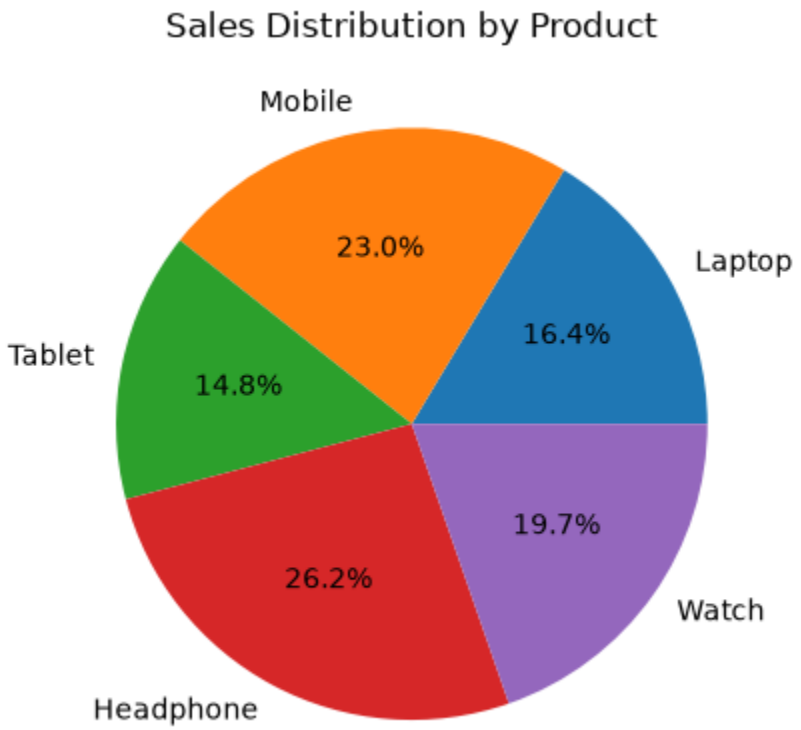
```
# Pie Chart
# Shows the percentage contribution of each product

plt.pie(
    dataset["Sales"],
    labels=dataset["Product"],
    autopct="%1.1f%%"
)

plt.title("Sales Distribution by Product")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

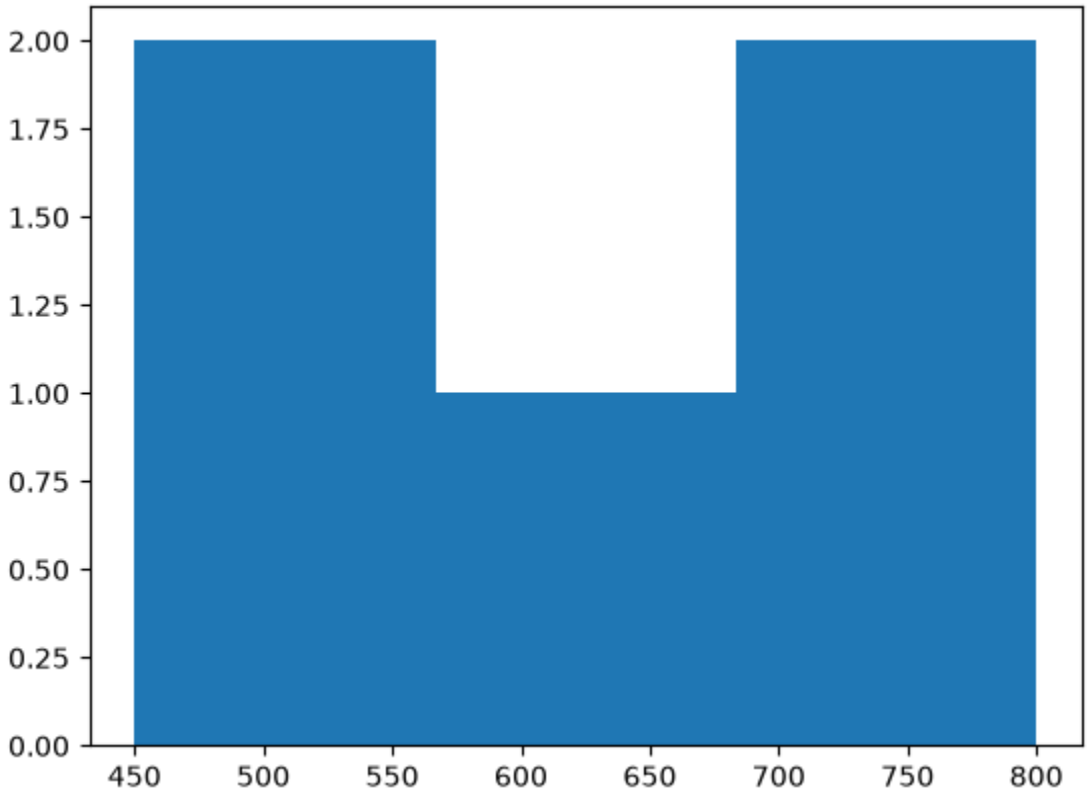


In [40]:

```
plt.hist(dataset["Sales"], bins=3)
```

Out[40]:

```
(array([2., 1., 2.]),
 array([450.        , 566.66666667, 683.33333333, 800.        ]),
 <BarContainer object of 3 artists>)
```



In [41]:

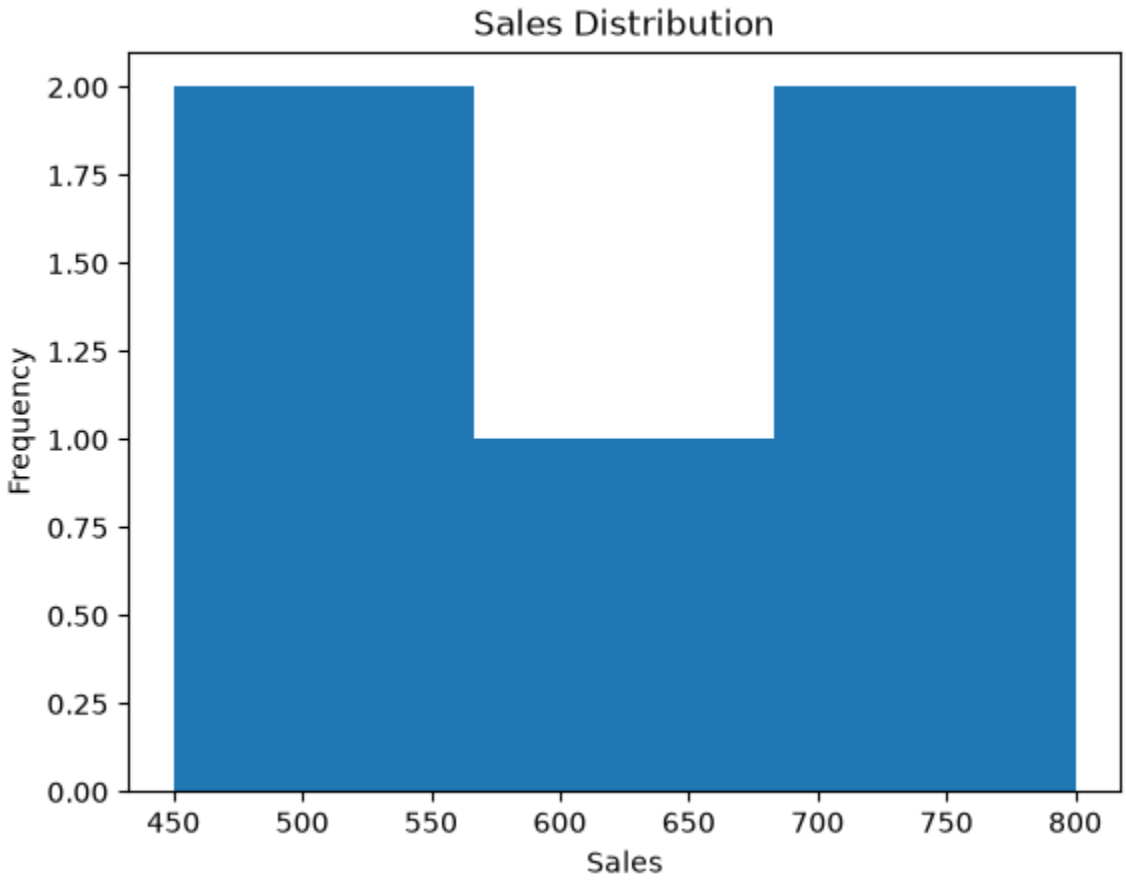
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Histogram
# Shows the distribution of sales values

plt.hist(
    dataset["Sales"],
    bins=3
)

plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.title("Sales Distribution")

plt.show()
```



In [42]:

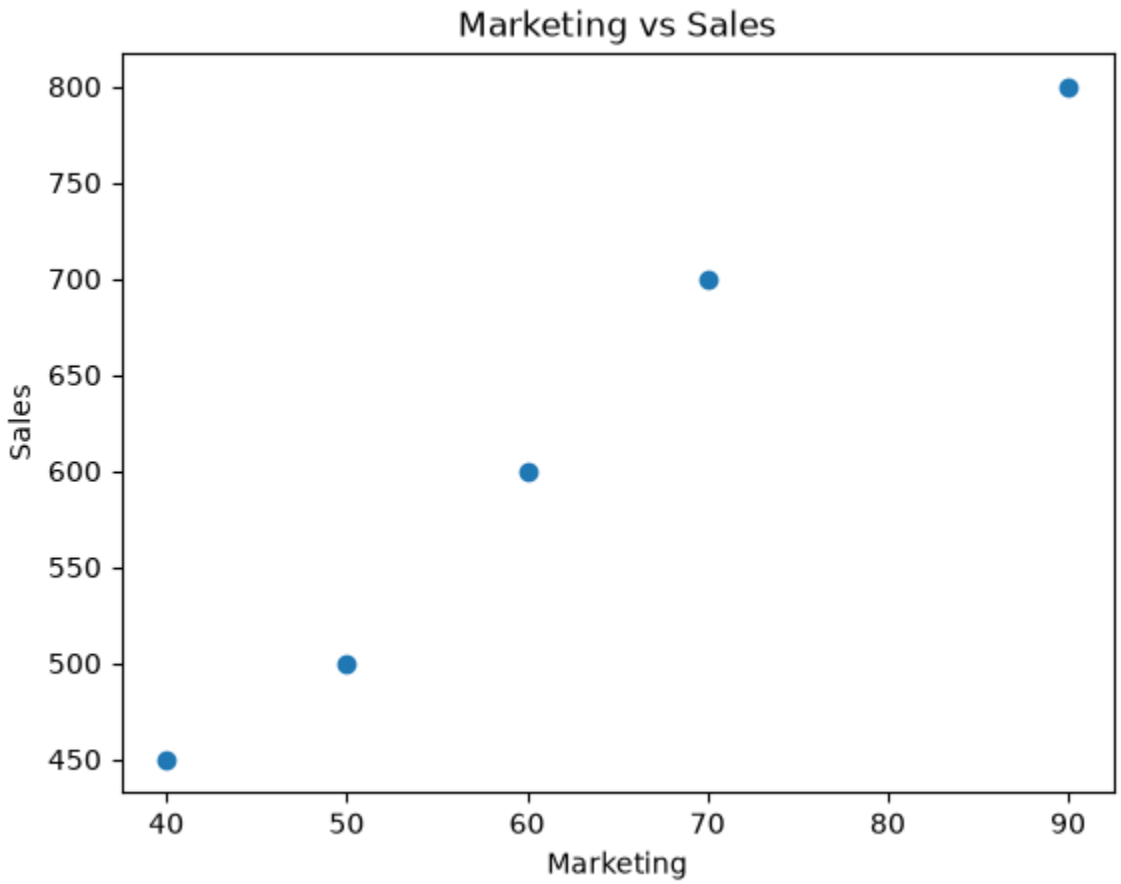
```
# Scatter Plot
# Shows relationship between Marketing and Sales

plt.scatter(
    dataset["Marketing"],
    dataset["Sales"]
)

plt.xlabel("Marketing")
plt.ylabel("Sales")
plt.title("Marketing vs Sales")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [43]:

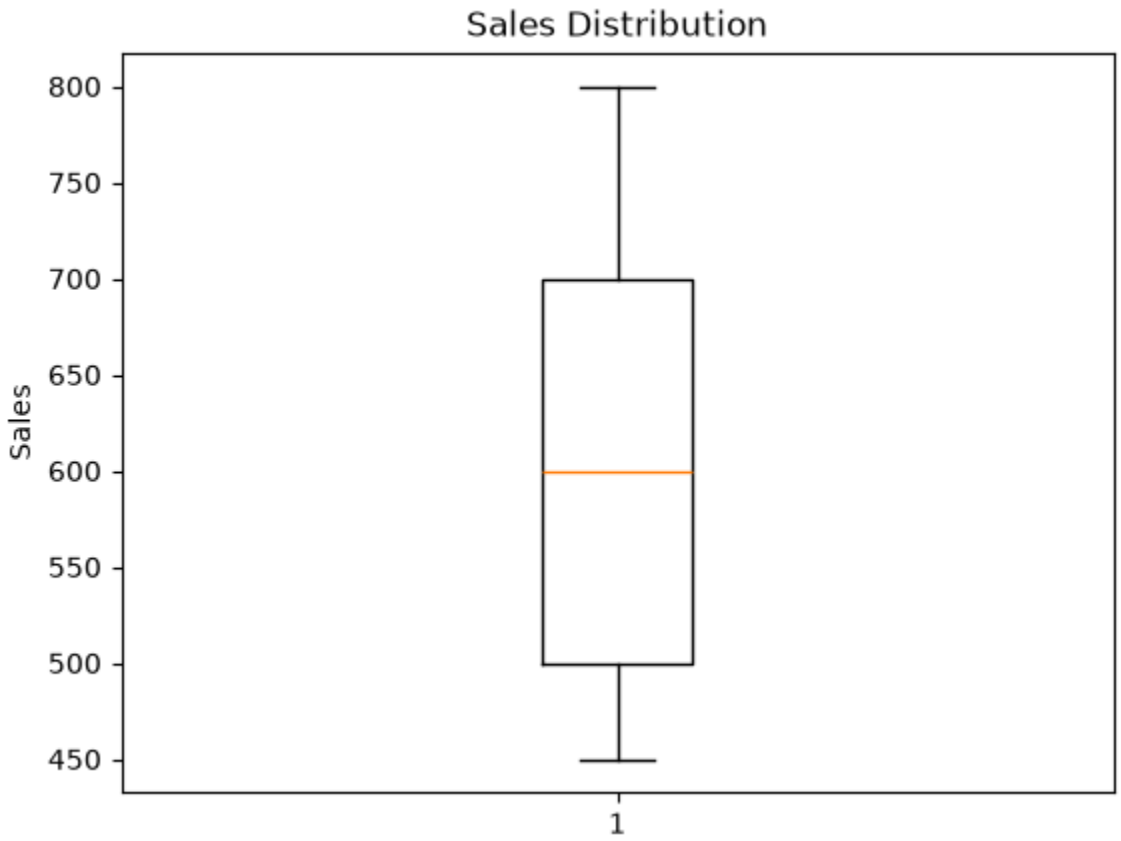
```
# Box Plot
# Shows the distribution and possible outliers in Sales

plt.boxplot(dataset["Sales"])

plt.ylabel("Sales")
plt.title("Sales Distribution")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [44]:

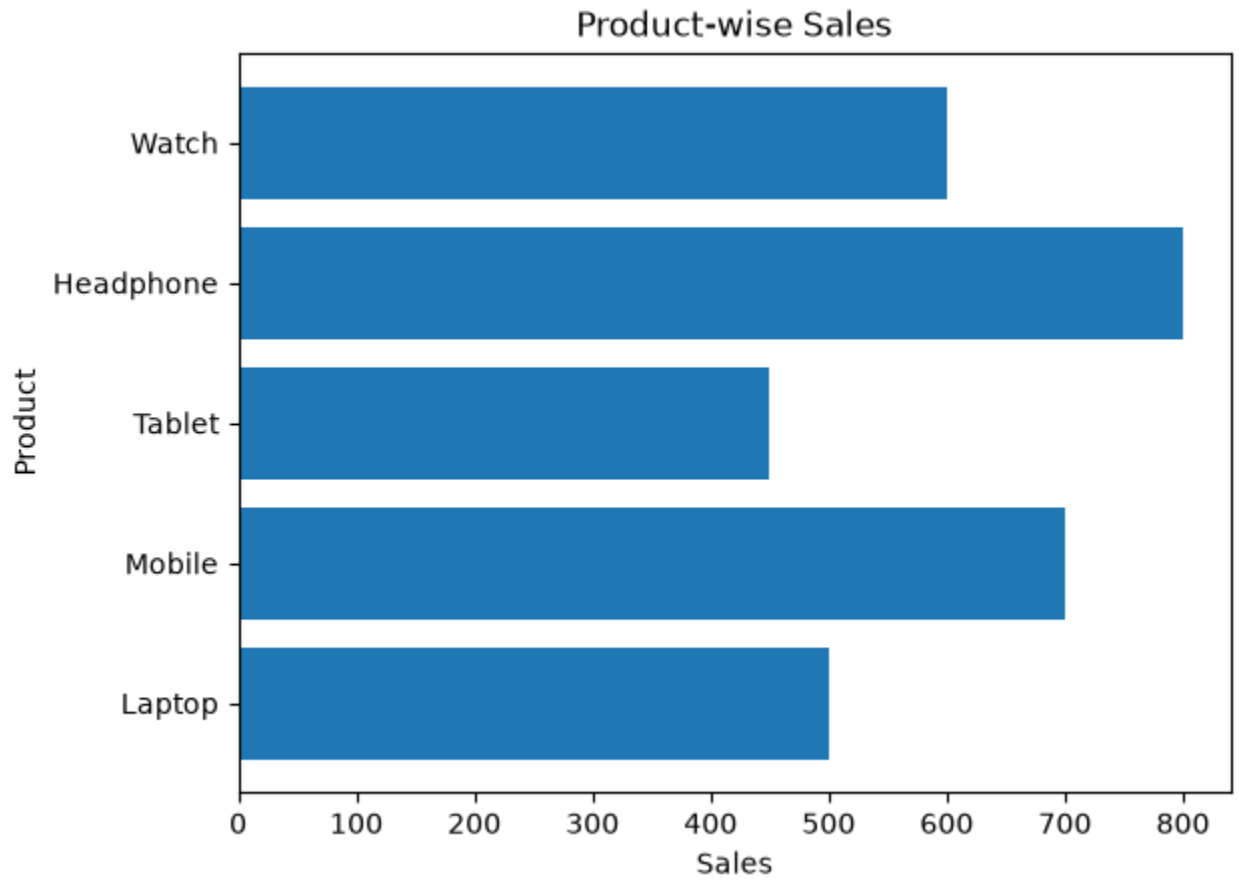
```
# Horizontal Bar Chart

plt.barh(
    dataset["Product"],
    dataset["Sales"]
)

plt.xlabel("Sales")
plt.ylabel("Product")
plt.title("Product-wise Sales")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [45]:

```
# Area Chart

x = range(len(dataset["Month"]))

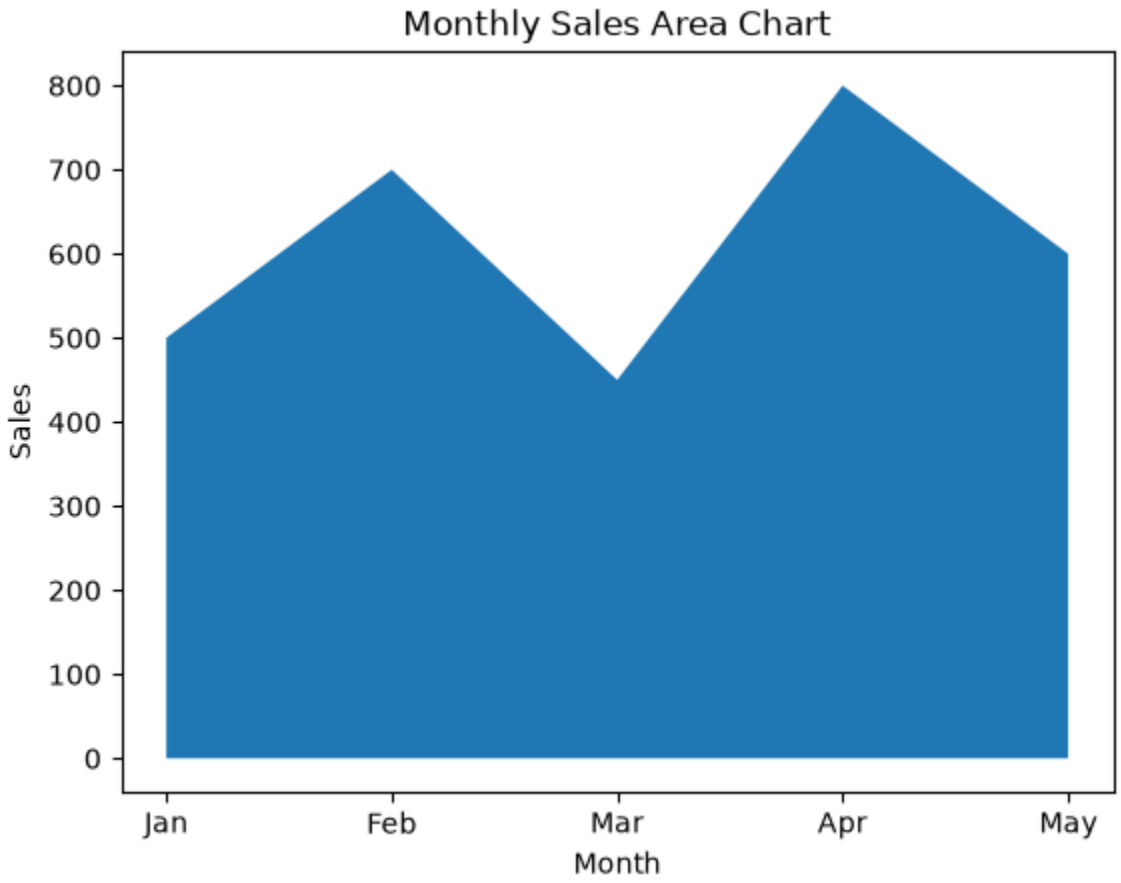
plt.fill_between(
    x,
    dataset["Sales"]
)

plt.xticks(
    x,
    dataset["Month"]
)

plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales Area Chart")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [46]:

```
# Stack Plot
# Shows Sales and Profit together

x = range(len(dataset["Month"]))

plt.stackplot(
    x,
    dataset["Sales"],
    dataset["Profit"],
    labels=["Sales", "Profit"]
)

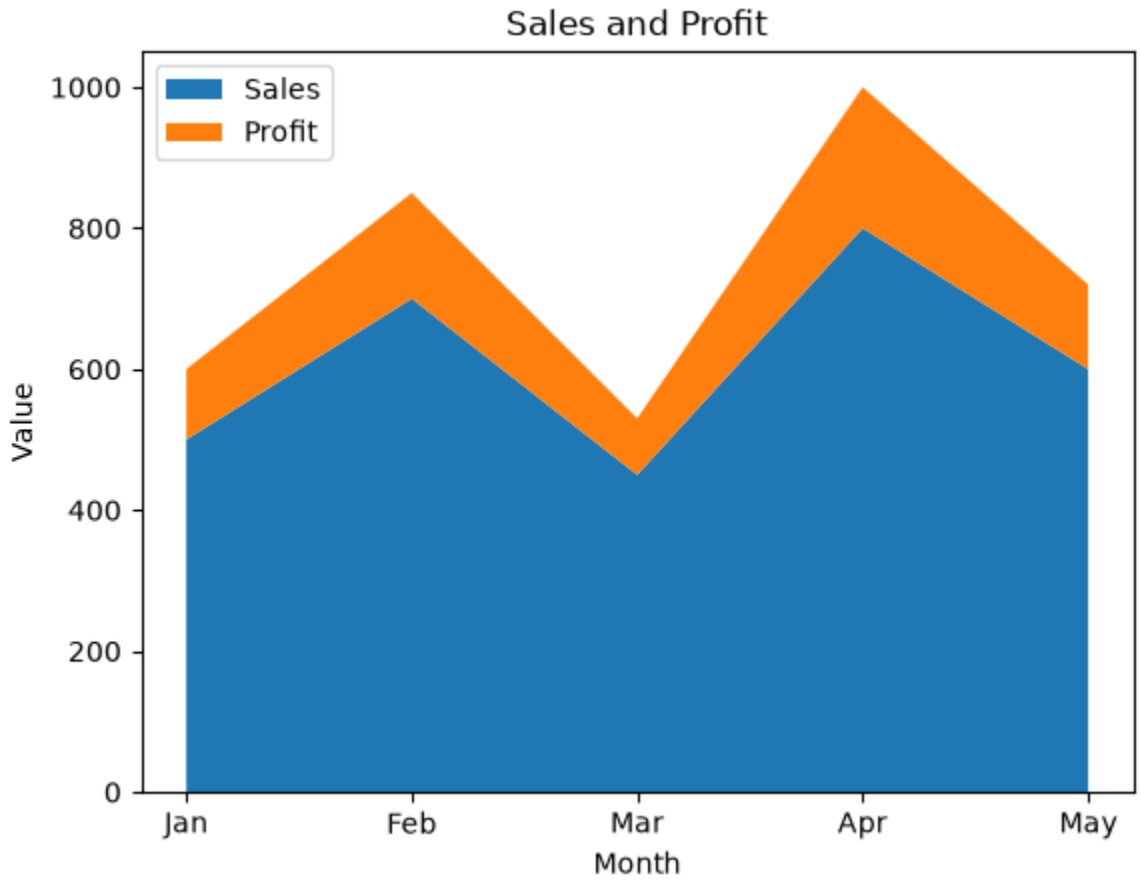
plt.xticks(
    x,
    dataset["Month"]
)

plt.xlabel("Month")
plt.ylabel("Value")
plt.title("Sales and Profit")

plt.legend()

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [47]:

```
# Create data for heatmap

heatmap = [
    dataset["Sales"],
    dataset["Profit"],
    dataset["Marketing"]
]
```

In [48]:

```
# Heatmap

plt.imshow(heatmap)

plt.xticks(
    range(5),
    dataset["Month"]
)

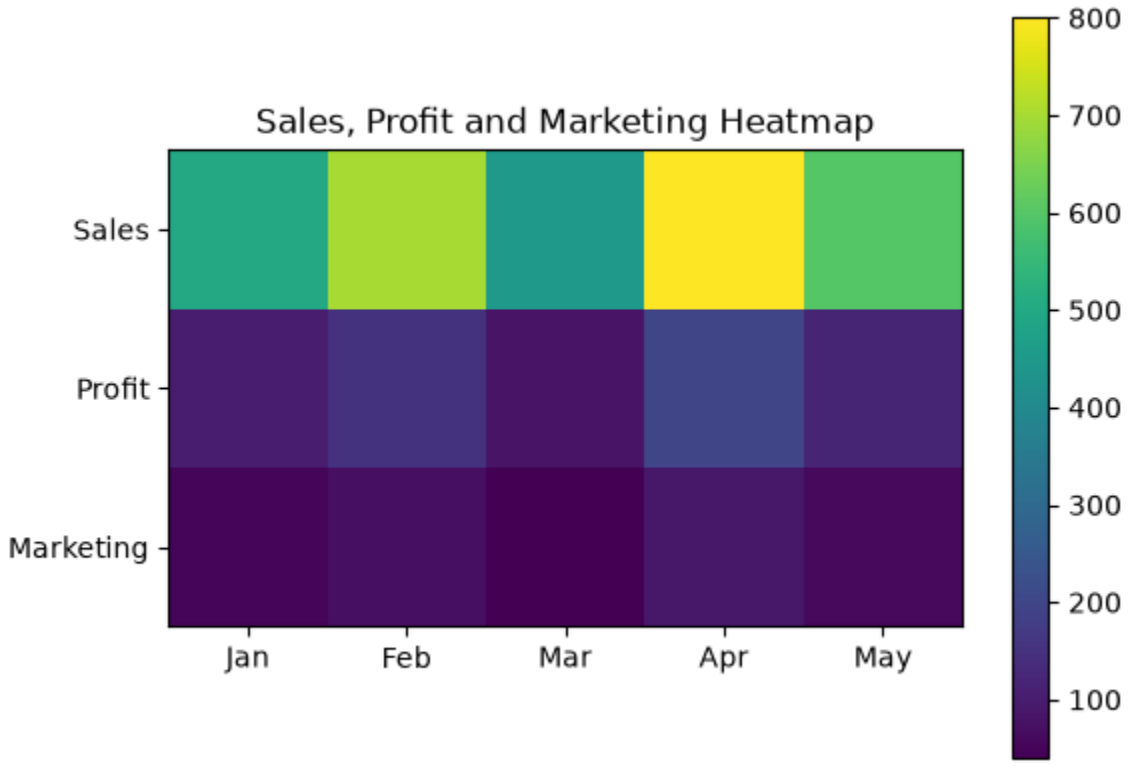
plt.yticks(
    range(3),
    ["Sales", "Profit", "Marketing"]
)

plt.colorbar()

plt.title("Sales, Profit and Marketing Heatmap")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



In [ ]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF